

Kubernetes CheatSheet - Modul 347

Inhaltsverzeichnis

- [Kubernetes CheatSheet - Modul 347](#)
 -  [Inhaltsverzeichnis](#)
 -  [Grundkonzepte](#)
 - Was ist Kubernetes
 - Cluster & API
 - Ressourcen & Controller
 -  [Kernkomponenten](#)
 - Pod
 - Deployment
 - Service
 - Zusammenspiel
 -  [YAML-Manifeste](#)
 - Pod-Manifest
 - Deployment-Manifest
 - Service-Manifest
 -  [kubectl Befehle](#)
 - Erstellen & Anwenden
 - Anzeigen & Überwachen
 - Debugging & Logs
 - Löschen & Verwalten
 -  [Labels & Selektoren](#)
 - Label-Konzepte
 - Selektor-Syntax
 -  [Netzwerk & Services](#)
 - ClusterIP
 - LoadBalancer
 - Port-Forwarding
 -  [Ressourcenverwaltung](#)
 - Controller-Prinzip
 - Ausfallsicherheit
 -  [Prüfungsrelevante Konzepte](#)
 - DNS & Service Discovery
 - Label-Selektoren in Deployments
 - Service → Pod Verbindung
 - Wichtige Unterschiede
 - Typische Prüfungsfragen
 - Kubectl Kurzformen
 - Häufige Befehle für Prüfung

Grundkonzepte

Was ist Kubernetes

Kubernetes ist eine Plattform für:

- Betrieb von Containern
- Automatisches Ausrollen von Updates
- Aufrechterhaltung von Service-Levels
- Bedarfsgerechte Skalierung
- Sicherung des Zugriffs
- Selbstheilende Anwendungen

Cluster & API

Cluster:

- Satz einzelner Server (Knoten)
- Alle mit Container-Laufzeit (Docker) konfiguriert
- Zu einer logischen Einheit verbunden
- Führt Ihre Anwendungen aus

API:

- Kubernetes-API zur Definition von Anwendungen
- YAML-Dateien werden an API gesendet
- API vergleicht gewünschten vs. aktuellen Zustand
- Nimmt erforderliche Änderungen vor

Ressourcen & Controller

Ressourcen:

- Pod, Deployment, Service, etc.
- Definiert in YAML-Manifesten
- Haben gewünschten Zustand

Controller:

- Verwalten andere Ressourcen
- Wichtigster: Deployment (verwaltet Pods)
- Sorgen für gewünschten Zustand
- Erstellen/Löschen Ressourcen automatisch

Kernkomponenten

Pod

Was ist ein Pod:

- Kleinste ausführbare Einheit
- Umhüllt Container in virtualisierte Umgebung
- Läuft auf einem einzelnen Knoten
- Hat eigene IP-Adresse
- Kann einen oder mehrere Container enthalten

Pod-Eigenschaften:

- Teilen sich localhost-Adresse
- Kommunizieren über localhost
- Vergängliche IP-Adressen
- Werden von Deployments verwaltet

Deployment

Was ist ein Deployment:

- Controller für Pods
- Definiert gewünschte Anzahl Replikas
- Verwaltet Pod-Lebenszyklus
- Rollout-Management
- Ausfallsicherheit

Deployment vs. Pod:

- **Pod:** Direkt erstellt, keine Ausfallsicherheit
- **Deployment:** Verwaltet Pods, automatischer Neustart, Skalierung

Service

Was ist ein Service:

- Netzwerk-Abstraction für Pods
- Stabile IP-Adresse und DNS-Name
- Load Balancing zwischen Pods
- Service Discovery im Cluster

Service-Typen:

- **ClusterIP:** Nur cluster-intern erreichbar
- **LoadBalancer:** Extern erreichbar

Zusammenspiel

Service → Deployment → Pod(s) → Container(s)

1. **Service** leitet Traffic weiter
2. **Deployment** verwaltet gewünschte Pod-Anzahl
3. **Pods** führen Container aus
4. **Container** laufen Anwendungen

YAML-Manifeste

Pod-Manifest

```
apiVersion: v1
kind: Pod
metadata:
  name: hello-kubernetes-3
  labels:
    app: hello-app
spec:
  containers:
    - name: web
      image: mainho/hello-kubernetes
```

Struktur:

- **apiVersion:** API-Version (v1 für Pods)
- **kind:** Ressourcentyp (Pod)
- **metadata:** Name und Labels
- **spec:** Container-Spezifikation

Deployment-Manifest

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-kubernetes-4
spec:
  replicas: 3
  selector:
    matchLabels:
      app: hello-kubernetes-4
  template:
    metadata:
      labels:
        app: hello-kubernetes-4
    spec:
      containers:
        - name: web
          image: mainho/hello-kubernetes
```

Wichtige Felder:

- **replicas**: Gewünschte Pod-Anzahl
- **selector.matchLabels**: Welche Pods gehören zum Deployment
- **template**: Pod-Vorlage für neue Pods

Service-Manifest

```
apiVersion: v1
kind: Service
metadata:
  name: numbers-api
spec:
  ports:
    - port: 80
      targetPort: 80
  selector:
    app: numbers-api
  type: ClusterIP
```

Service-Felder:

- **ports**: Port-Mapping
- **selector**: Welche Pods erhalten Traffic
- **type**: Service-Typ (ClusterIP, LoadBalancer)

🔑 kubectl Befehle

Erstellen & Anwenden

```
# Aus YAML-Datei erstellen
kubectl apply -f pod.yaml
kubectl apply -f deployment.yaml

# Direkt erstellen
kubectl run hello-pod --image=nginx
kubectl create deployment hello-deploy --image=nginx

# Aus URL anwenden
kubectl apply -f https://raw.githubusercontent.com/...
```

Anzeigen & Überwachen

```
# Pods anzeigen
kubectl get pods
kubectl get pods -o wide
kubectl get pods -l app=myapp

# Deployments anzeigen
kubectl get deployments
kubectl get deploy

# Services anzeigen
kubectl get services
kubectl get svc

# Alle Ressourcen
kubectl get all

# Details anzeigen
kubectl describe pod <pod-name>
kubectl describe deployment <deploy-name>

# Custom Columns
kubectl get pods -o custom-columns=NAME:metadata.name,IP:status.podIP
```

Debugging & Logs

```
# Logs anzeigen
kubectl logs <pod-name>
kubectl logs --tail=10 <pod-name>
kubectl logs -f <pod-name> # Follow
kubectl logs -l app=myapp # Nach Label

# In Pod einsteigen
kubectl exec -it <pod-name> -- sh
kubectl exec -it <pod-name> -- bash

# Port-Forwarding
kubectl port-forward pod/<pod-name> 8080:80
kubectl port-forward deploy/<deploy-name> 8080:80

# Dateien kopieren
kubectl cp <pod-name>:/path/to/file ./local-file
kubectl cp ./local-file <pod-name>:/path/to/file
```

Löschen & Verwalten

```
# Einzelne Ressourcen löschen
kubectl delete pod <pod-name>
kubectl delete deployment <deploy-name>
kubectl delete service <service-name>

# Nach Label löschen
kubectl delete pods -l app=myapp

# Alle Ressourcen eines Typs
kubectl delete pods --all
kubectl delete deployments --all

# Aus Datei löschen
kubectl delete -f deployment.yaml
```

Labels & Selektoren

Label-Konzepte

Labels sind Key-Value-Paare:

```
metadata:
  labels:
    app: hello-kubernetes-4
    version: v1
    environment: production
```

Verwendung:

- Identifikation von Ressourcen
- Gruppierung verwandter Objekte
- Selektion für Services/Deployments

Selektor-Syntax

```
# Pods mit bestimmtem Label
kubectl get pods -l app=myapp
kubectl get pods -l version=v1

# Mehrere Labels (AND)
kubectl get pods -l app=myapp,version=v1

# Label existiert
kubectl get pods -l app

# Label ändern
kubectl label pods <pod-name> version=v2
kubectl label pods <pod-name> --overwrite app=newname

# Labels anzeigen
kubectl get pods --show-labels
```

Netzwerk & Services

ClusterIP

Standard Service-Typ:

- Nur cluster-intern erreichbar
- Stabile IP-Adresse
- DNS-Name im Cluster
- Load Balancing zwischen Pods

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: ClusterIP # Standard, kann weggelassen werden
  ports:
    - port: 80
      targetPort: 8080
  selector:
    app: my-app
```

LoadBalancer

Für externen Zugriff:

- Externe IP-Adresse
- Traffic von außen möglich
- Automatisches Port-Mapping

```
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: LoadBalancer
  ports:
    - port: 8080
      targetPort: 80
  selector:
    app: web-app
```

Port-Forwarding

Temporärer Zugriff für Testing:

```
# Pod direkt
kubectl port-forward pod/mypod 8080:80

# Über Deployment
kubectl port-forward deploy/myapp 8080:80

# Über Service
kubectl port-forward svc/myervice 8080:80
```

Zugriff dann über: <http://localhost:8080>

Ressourcenverwaltung

Controller-Prinzip

Deployment Controller:

1. Überwacht gewünschten Zustand (replicas: 3)
2. Vergleicht mit aktuellem Zustand
3. Erstellt/löscht Pods bei Bedarf
4. Reagiert auf Pod-Ausfälle automatisch

Beispiel:

- Deployment will 3 Pods
- Ein Pod stirbt → nur noch 2 Pods
- Controller erstellt automatisch neuen Pod
- Wieder 3 Pods = gewünschter Zustand

Ausfallsicherheit

Pod stirbt:

- Deployment erkennt dies
- Startet automatisch Ersatz-Pod
- Neue IP-Adresse für neuen Pod
- Service leitet Traffic weiter

Container stirbt:

- Kubernetes startet Container neu
- Pod-IP bleibt gleich
- Automatische Wiederherstellung

💡 Prüfungsrelevante Konzepte

DNS & Service Discovery

```
# Pod kann anderen Service erreichen über:
wget -O - http://service-name
ping service-name

# DNS funktioniert nur für Services, nicht für Pods
```

Label-Selektoren in Deployments

```
# Deployment verwaltet Pods mit matching Labels
spec:
  selector:
    matchLabels:
      app: hello-kubernetes-4 # Muss mit Pod-Labels übereinstimmen
  template:
    metadata:
      labels:
        app: hello-kubernetes-4 # Pod bekommt diese Labels
```

Service → Pod Verbindung

```
# Service wählt Pods über Labels aus
apiVersion: v1
kind: Service
metadata:
  name: numbers-api
spec:
  selector:
    app: numbers-api # Leitet Traffic an Pods mit diesem Label
```

Wichtige Unterschiede

Aspekt	Pod	Deployment
Erstellung	Direkt	Controller verwaltet
Ausfallsicherheit	Keine	Automatischer Neustart
Skalierung	Manuell	Automatisch/Deklarativ
Updates	Manuell ersetzen	Rolling Updates
Verwendung	Testing/Debugging	Produktion

Typische Prüfungsfragen

1. "Warum stirbt Pod nicht automatisch neu?"

- Pod hat keinen Controller (Deployment)

2. "Service erreicht Pods nicht?"

- Label-Selector prüfen
- Pod-Labels vs. Service-Selector

3. "Unterschied ClusterIP vs LoadBalancer?"

- ClusterIP: nur intern
- LoadBalancer: extern erreichbar

4. "YAML-Manifest erklären?"

- apiVersion, kind, metadata, spec
- Labels und Selektoren
- Container-Definition

Kubectl Kurzformen

```
# Abkürzungen
po = pods
deploy = deployments
svc = services
```

Häufige Befehle für Prüfung

```
# Status prüfen
kubectl get pods
kubectl get deployments
kubectl get services

# Details anzeigen
kubectl describe pod <name>

# Labels anzeigen
kubectl get pods --show-labels
kubectl get pods -l app=myapp

# Port-Forward für Testing
kubectl port-forward deploy/myapp 8080:80

# Logs für Debugging
kubectl logs <pod-name>
```